프로토콜 버퍼 설정 분석

개요

이 문서는 Mars 블록체인 프로젝트의 프로토콜 버퍼(Protocol Buffer) 설정과 코드 생성 과정을 상세히 분석합니다. 프로토콜 버퍼는 gRPC 서비스와 메시지 타입을 정의하고, 다양한 언어로 코드를 자동 생성하는 데 사용됩니다.

파일 구조

```
proto/
├── buf.yaml # Buf CLI 메인 설정 파일
├── buf.lock # 의존성 잠금 파일
├── buf.gen.gogo.yaml # Go 코드 생성 설정
├── buf.gen.sta.yaml # OpenAPI 스펙 생성 설정 (Static)
├── buf.gen.swagger.yaml # Swagger 문서 생성 설정
├── buf.gen.ts.yaml # TypeScript 코드 생성 설정
└── mars/
└── mars/
├── module/ # 모듈 정의 proto 파일들
└── v1/ # API 버전 1 proto 파일들
```

buf.yaml 설정 분석

1. 버전 및 모듈 설정

```yaml
version: v2  # Buf CLI v2 사용
modules:
- path: proto  # proto 디렉토리를 모듈로 설정
```

2. 의존성 관리 (deps)

```yaml
deps:
- buf.build/cosmos/cosmos-proto  # Cosmos 기본 프로토 타입
- buf.build/cosmos/cosmos-sdk  # Cosmos SDK 프로토 정의
- buf.build/cosmos/gogo-proto  # GoGo Protobuf 확장
- buf.build/cosmos/ics23 # ICS23 머클 증명 스펙
- buf.build/googleapis/googleapis # Google APIs 프로토 정의
- buf.build/protocolbuffers/wellknowntypes # 표준 프로토 타입
- buf.build/cosmos/ibc # IBC 프로토콜 정의
```

각 의존성의 역할:

- **cosmos-proto**: Cosmos 생태계의 기본 프로토콜 정의
- **cosmos-sdk**: Cosmos SDK의 모든 모듈 프로토 정의 (bank, auth, staking 등)
- **gogo-proto**: Go 언어 최적화된 프로토콜 버퍼 확장
- **ics23**: 머클 트리 증명을 위한 ICS23 스펙 구현
- **googleapis**: Google의 표준 API 프로토 정의 (HTTP 어노테이션 등)
- **wellknowntypes**: 표준 프로토콜 버퍼 타입 (Any, Timestamp 등)

- **ibc**: Inter-Blockchain Communication 프로토콜 정의

3. 린팅 설정 (lint)

```yaml
lint:
disallow_comment_ignores: true # 주석으로 린트 무시 비허용
use:
- COMMENTS # 주석 필수
- STANDARD # 표준 스타일 가이드
- FILE_LOWER_SNAKE_CASE # 파일명 snake_case 강제
except:
- COMMENT_FIELD # 필드 주석 예외
- RPC_REQUEST_STANDARD_NAME # RPC 요청 이름 표준 예외
- RPC_RESPONSE_STANDARD_NAME # RPC 응답 이름 표준 예외
- SERVICE_SUFFIX # 서비스 접미사 예외
ignore:
- proto/tendermint # Tendermint 프로토 파일 무시
```

4. 호환성 검사 설정 (breaking)

```yaml
breaking:
use:
- FILE # 파일 레벨 호환성 검사
except:
```

```
- EXTENSION_NO_DELETE # 확장 삭제 허용

- FIELD_SAME_DEFAULT # 동일 기본값 필드 변경 허용
```

buf.lock 분석

buf.lock 파일은 의존성의 정확한 버전을 고정하여 재현 가능한 빌드를 보장합니다.

주요 의존성 버전 정보:

- **cosmos-proto**: commit `04467658e59e44bbb22fe568206e1f70`
- **cosmos-sdk**: commit `650cd9ad7f7a468e8e19975269958658`
- **gogo-proto**: commit `88ef6483f90f478fb938c37dde52ece3`
- **ibc**: commit `857539074f9c4dda961f9db5dcf8e462`

각 의존성은 commit hash와 digest를 통해 무결성이 보장됩니다.

코드 생성 설정 분석

1. buf.gen.gogo.yaml - Go 코드 생성

```yaml
plugins:

# GoGo Protobuf를 사용한 Go 코드 생성

- local: ["go", "tool", "github.com/cosmos/gogoproto/protoc-gen-gocosmos"]

out: .

opt:

- plugins=grpc # gRPC 서비스 코드 생성
```

```
# 타입 매핑 설정
- Mgoogle/protobuf/any.proto=github.com/cosmos/gogoproto/types/any
- Mcosmos/orm/v1/orm.proto=cosmossdk.io/orm
- Mcosmos/app/v1alpha1/module.proto=cosmossdk.io/api/cosmos/app/v1alpha1
# gRPC Gateway를 위한 HTTP 핸들러 생성
- local: ["go", "tool", "github.com/grpc-ecosystem/grpc-gateway/protoc-gen-grpc-
out: .
opt:
- logtostderr=true
- allow_colon_final_segments=true
```

생성되는 파일들:

- `*.pb.go` : 메시지 구조체와 직렬화 코드
- `*_grpc.pb.go` : gRPC 서비스 인터페이스와 클라이언트
- `*.pb.gw.go` : HTTP REST API 게이트웨이 코드

2. buf.gen.sta.yaml - OpenAPI 스펙 생성 (Static)

```
plugins:
- local: ["go", "tool", "github.com/grpc-ecosystem/grpc-gateway/v2/protoc-gen-op
out: .
opt:
- logtostderr=true
- openapi_naming_strategy=simple # 단순한 이름 전략
```

```
- ignore_comments=true # 주석 무시
- simple_operation_ids=false # 복잡한 operation ID 사용
- json_names_for_fields=false # proto 필드명 사용
```

3. buf.gen.swagger.yaml - Swagger 문서 생성

```yaml
plugins:
- local: ["go", "tool", "github.com/grpc-ecosystem/grpc-gateway/v2/protoc-gen-op
out: .
opt:
- logtostderr=true
- openapi_naming_strategy=fqn # 완전한 이름 사용
- json_names_for_fields=false
- generate_unbound_methods=true # 바인딩되지 않은 메서드도 생성
```

4. buf.gen.ts.yaml - TypeScript 코드 생성

```yaml
managed:
enabled: true # 관리 모드 활성화
plugins:
- remote: buf.build/community/stephenh-ts-proto
out: .
opt:
```

```
- logtostderr=true
- allow_merge=true # 파일 병합 허용
- json_names_for_fields=false
- ts_proto_opt=snakeToCamel=true # snake_case를 camelCase로 변환
- ts_proto_opt=esModuleInterop=true
- ts_proto_out=.
```

코드 생성 과정

1. Ignite CLI를 통한 생성

```
ignite generate proto-go --yes
```

이 명령어는 다음 과정을 수행합니다:

1. **의존성 해결**: buf.lock에 정의된 의존성들을 다운로드
2. **proto 파일 검증**: buf.yaml의 lint 규칙에 따라 proto 파일 검증
3. **코드 생성**: 각 buf.gen.*.yaml 설정에 따라 코드 생성
4. **출력 파일 배치**: 생성된 파일들을 적절한 위치에 배치

2. 생성되는 출력 파일들

Go 코드 (buf.gen.gogo.yaml)

```
x/mars/types/
├── tx.pb.go # 트랜잭션 메시지 구조체
```

```
├── tx_grpc.pb.go # gRPC 서비스 인터페이스
├── query.pb.go # 쿼리 메시지 구조체
├── query_grpc.pb.go # 쿼리 gRPC 서비스
├── genesis.pb.go # 제네시스 상태 구조체
└── *.pb.gw.go # HTTP REST 게이트웨이
```

API 문서 (buf.gen.swagger.yaml)

```
docs/static/
└── openapi.yml # OpenAPI 3.0 스펙 파일
```

TypeScript 코드 (buf.gen.ts.yaml)

```
ts-client/
├── mars.mars.v1/ # Mars 모듈 TypeScript 클라이언트
└── index.ts # 통합 인덱스 파일
```

개발 워크플로우

1. Proto 파일 수정 시

1. `proto/mars/mars/v1/*.proto` 파일 수정
2. `make proto-gen` 또는 `ignite generate proto-go --yes` 실행
3. 생성된 Go 코드 확인 및 테스트

4. API 문서 업데이트 확인

2. 새로운 메시지/서비스 추가 시

1. proto 파일에 새로운 메시지나 서비스 정의
2. buf lint 규칙 준수 확인
3. 코드 재생성
4. 구현 코드 작성 (keeper, handler 등)

3. 호환성 관리

- buf breaking 체크를 통해 API 호환성 검증
- 버전 업그레이드 시 마이그레이션 계획 수립
- 클라이언트 코드 업데이트 가이드 제공

모범 사례

1. Proto 파일 작성

- 명확하고 일관된 네이밍 컨벤션 사용
- 적절한 주석과 문서화
- 필드 번호 관리 (삭제된 필드 번호 재사용 금지)

2. 의존성 관리

- buf.lock 파일을 버전 관리에 포함
- 정기적인 의존성 업데이트 및 테스트
- 보안 취약점 모니터링

3. 코드 생성 최적화

- 필요한 언어/형식만 생성하여 빌드 시간 단축
- 생성된 파일은 버전 관리에서 제외 (.gitignore)
- CI/CD 파이프라인에서 자동 생성 및 검증